Predicting Loan Default Risk with GBDT and MLP

Columbia COMS4995 Assignment 2

Dipakshi Pal
Columbia University
dp3245@columbia.edu

Code: Click here to view Github Repository

1 Introduction

1.1 Problem Definition

One of the most important factors considered when someone applies for a loan is a credit risk assessment, which consists of using application, credit, and behavioral data to determine the likelihood of the individual paying back the loan. Therefore, it is important to develop accurate and reliable predictive models in classifying loan applicants to minimize financial losses. In this assignment, we will explore and compare two machine learning approaches: Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP) to conduct this analysis and evaluate their effectiveness in classification.

1.2 Dataset Description

The chosen dataset is the Home Credit Default Risk dataset from Kaggle (Link). It consists of 10 files, but I am focusing on the application_train.csv file for this assignment. This file contains 307,511 loan applications with 122 features spanning demographic information, financial history, credit bureau data, and behavioral indicators. The target variable is binary: 0 indicating the applicant repaid the loan and 1 indicating default. The dataset is notably imbalanced, with approximately 91.9% non-defaults and 8.1% defaults, which has direct implications for model selection and evaluation.

2 Data Preparation

2.1 Missing Values

Identifying if there are missing values in a dataset is key because empty or null values can lead to incomplete patterns and biased results. To systematically identify missing values in the given dataset, I created a summary table, as seen in Figure 1, that included the number of missing values, the total count of each variable, and the missing percentage of each variable.

In this table, I noticed there were a large number of columns missing more than 50% of their values, which could become hard to accurately impute. Therefore, I decided to examine the specific columns more intently, as seen in Figure 2. This revealed that many of the columns, specifically those describing the applican't residential building, contained repeated information in three different ways. For example, there is a LANDAREA_AVG, LANDAREA_MEDI, and LANDAREA_MODE. All three of these columns are conveying the same general information about the land area of the applicant's building, simply aggregated as the average, median, and mode across units in the building respectively. Since these columns are redundant and missing the majority of the data, keeping all three would introduce noise to the dataset and not add anything meaningful to signal. Therefore, I decided to drop the columns ending in "_AVG" or "_MODE" and only keep the "_MEDI" to be a single representative measure for each building attribute.

Another column I dropped was the SK_ID_CURR column since it is not a feature, and rather just an identifying marker. For the purpose of this assignment goal, it was not necessary.

Ultimately, this resulted in the dropping of 34 columns, leaving 88 columns for the rest of the project.

	missing_count	total_count	missing_%
SK_ID_CURR	0	307511	0.00
TARGET	0	307511	0.00
NAME_CONTRACT_TYPE	0	307511	0.00
CODE_GENDER	0	307511	0.00
FLAG_OWN_CAR	0	307511	0.00
FLAG_OWN_REALTY	0	307511	0.00
CNT_CHILDREN	0	307511	0.00
AMT_INCOME_TOTAL	0	307511	0.00
AMT_CREDIT	0	307511	0.00
AMT_ANNUITY	12	307499	0.00
AMT_GOODS_PRICE	278	307233	0.09
NAME_TYPE_SUITE	1292	306219	0.42
NAME_INCOME_TYPE	0	307511	0.00
NAME_EDUCATION_TYPE	0	307511	0.00
NAME_FAMILY_STATUS	0	307511	0.00
NAME_HOUSING_TYPE	0	307511	0.00
REGION_POPULATION_RELATIVE	0	307511	0.00
DAYS_BIRTH	0	307511	0.00
DAYS_EMPLOYED	0	307511	0.00
DAYS_REGISTRATION	0	307511	0.00
DAYS_ID_PUBLISH	0	307511	0.00
OWN_CAR_AGE	202929	104582	65.99
FLAG_MOBIL	0	307511	0.00
FLAG_EMP_PHONE	0	307511	0.00
FLAG_WORK_PHONE	0	307511	0.00
FLAG_CONT_MOBILE	0	307511	0.00
FLAG_PHONE	0	307511	0.00
FLAG_EMAIL	0	307511	0.00
OCCUPATION_TYPE	96391	211120	31.35
CNT_FAM_MEMBERS	2	307509	0.00

Figure 1: Snippet of Combined Model Evaluation Results

```
COLUMNS MISSING >50% OF VALUES:
 OWN_CAR_AGE (65.99%)
 EXT_SOURCE_1 (56.38%)
 APARTMENTS_AVG (50.75%)
 BASEMENTAREA_AVG (58.52%)
 YEARS_BUILD_AVG (66.5%)
 COMMONAREA_AVG (69.87%)
 ELEVATORS_AVG (53.3%)
 ENTRANCES_AVG (50.35%)
 FLOORSMIN_AVG (67.85%)
 LANDAREA_AVG (59.38%)
 LIVINGAPARTMENTS_AVG (68.35%)
 LIVINGAREA_AVG (50.19%)
 NONLIVINGAPARTMENTS_AVG (69.43%)
 NONLIVINGAREA_AVG (55.18%)
 APARTMENTS_MODE (50.75%)
 BASEMENTAREA_MODE (58.52%)
 YEARS_BUILD_MODE (66.5%)
 COMMONAREA_MODE (69.87%)
 ELEVATORS_MODE (53.3%)
 ENTRANCES_MODE (50.35%)
 FLOORSMIN_MODE (67.85%)
 LANDAREA_MODE (59.38%)
 LIVINGAPARTMENTS_MODE (68.35%)
 LIVINGAREA_MODE (50.19%)
 NONLIVINGAPARTMENTS_MODE (69.43%)
 NONLIVINGAREA_MODE (55.18%)
 APARTMENTS_MEDI (50.75%)
 BASEMENTAREA_MEDI (58.52%)
 YEARS_BUILD_MEDI (66.5%)
 COMMONAREA_MEDI (69.87%)
 ELEVATORS_MEDI (53.3%)
 ENTRANCES_MEDI (50.35%)
 FLOORSMIN_MEDI (67.85%)
 LANDAREA_MEDI (59.38%)
 LIVINGAPARTMENTS_MEDI (68.35%)
 LIVINGAREA_MEDI (50.19%)
 NONLIVINGAPARTMENTS_MEDI (69.43%)
 NONLIVINGAREA_MEDI (55.18%)
 FONDKAPREMONT_MODE (68.39%)
 HOUSETYPE_MODE (50.18%)
 WALLSMATERIAL_MODE (50.84%)
```

Figure 2: Columns with >50% Missing Values

2.2 Split Data

To avoid data leakage during data imputation to account for missing values, the data was split so the imputation only occurs on the train set. This is crucial because imputing before on the entire dataset would allow information from the validation and test sets to leak into the training process, leading to unreliable, overly optimistic performance estimates.

The dataset was split using an 70-15-15 train/val/test split. This resulted in 215257 training samples, 46127 validation samples, and 46127 test samples.

Furthermore, stratified splitting was used to make sure both sets had the same class distribution as the original dataset. Figure 3 shows the splits within the train/val/test sets.

This train/val/test split was used across both models for the rest of this assignment to ensure fair comparison.

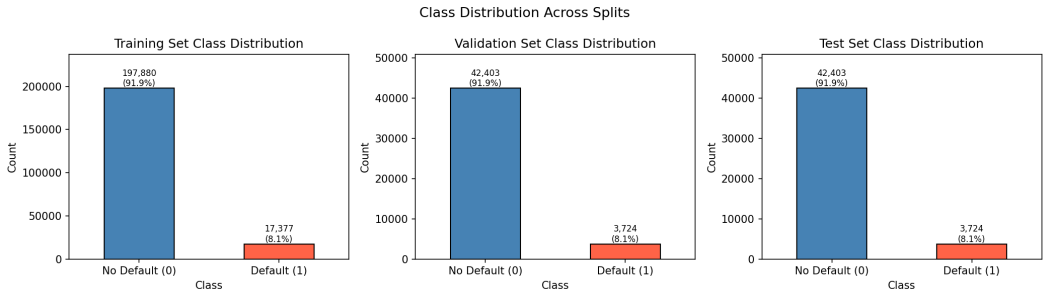

Figure 3: Train/Test Split

2.3 Noisy and Inconsistent Data

Noisy and inconsistent data, such as typos, random errors, incorrect units, or duplicates, can also distort analysis by adding variance to a dataset. Although it is harder to identify than missing values, it is equally important to check for these issues before training a model.

First, I started by getting a statistical summary of all the features, as shown in Figure 4, allowing for a quick scan of minimum and maximum values. The goal here was to quickly identify if there was anything physically impossible, such as a negative age.

One thing that stood out through this was the DAYS_EMPLOYED column which had a maximum value of 365,243 days, or 1,000 years of employment. This was obviously an error/outlier, as seen in Figure 4, so I replaced any instances of 365,243 with NaN so during imputation it is imputed.

Other than that, nothing looked out of place. However, just in case I also checked that there was no negative incomes, number of children, number of family members.

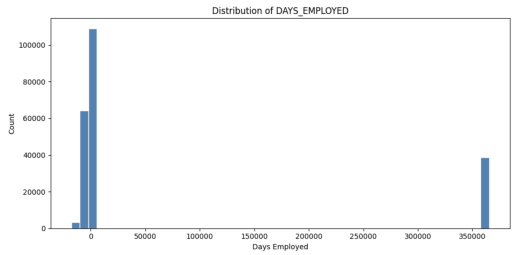

Figure 4: Distribution of DAYS_EMPLOYED

2.4 Imputation for Missing Values

Since the dataset still has missing values across several columns, it was important to impute after the split to ensure no information was lost during model training. All the imputation was implemented on the training set, then applied to the validation and test sets, to prevent data leakage.

For numerical (continuous) columns, median imputation was used. I chose to use median because after visualizing the splits for these columns, most of them were skewed, making the median a more robust measure compared to mean (which is more sensitive to extreme outliers).

On the other hand, for the categorical columns, I used mode imputation, where the missing values were replaced with the most frequently occurring category in the training set. This made sense to be the most natural imputation strategy for categorical data since mean and median are not defined here. Furthermore, looking at the distributions of categorical features, most columns had a clear dominant category, showcasing the mode serves as a reasonable and representative substitute for missing entries without introducing significant bias.

2.5 Feature Engineering

Beyond the raw features provided, I constructed several new features.

- Credit to Income
  - Compares how large a loan is relative to the individual's income and showcases debt burden.
  - Calculated by dividing AMT_CREDIT by AMT_INCOME_TOTAL.
  - A higher ratio suggests the individual is borrowing significantly more than they earn, which may indicate a greater likelihood of default.
- Annuity to Income
  - Reflects the monthly strain of an individual's finances, specifically what fraction of annual income goes towards loan repayments.
  - Calculated by dividing AMT_ANNUITY by AMT_INCOME_TOTAL.

  - Individuals spending a large share of their income on repayments have less financial flexibility, making them higher risk.

- Payment Rate
  - Reveals the length of the loan, in turn affecting how long an individual is exposed to repayment risks.
  - Calculated by dividing AMT_ANNUITY by AMT_CREDIT.
  - Lower value indicates a longer repayment horizon, which my suggest a higher cumulative default risk over time.

- Employment to Age
  - Showcases the idea that employment stability and an individual's age can provide insights into their financial maturity and job consistency.
  - Calculated by dividing DAYS_EMPLOYED by DAYS_BIRTH.
  - High ratio suggests more stable employment history, which generally suggests lower risk.

- Age Group
  - While AGE_YEARS captures age as a continuous value, binning it into categories helps capture the non-linear relationship between life stage and financial behavior.
  - Split into 4 categories: Young (<30), Middle (30-45), Senior (45-60), and Elder (60+)
  - Younger applicants may have shorter credit histories and less stable income, while older applicants may have more established financial patterns, making this a meaningful signal for default risk.

- Dependents
  - Reflects the financial impact of the number of dependents.
  - Split into 3 categories: None (0 children), Low (1-2 children), and High (3+ children).
  - Higher number of dependents will likely lower the amount of disposable income and lower an individual's ability to meet loan replayments.

- External Source Mean Score
  - EXT_SOURCE_1,EXT_SOURCE_2, and EXT_SOURCE_3 are external credit scores within the dataset.
  - They are the strongest individual predictors of default in this dataset.
  - Combining them allows for a more comprehensive score for the model to learn from.

2.6 One Hot Encoding

The categorical variables in the dataset needed to be converted because they contain integer values that have no numeric meaning (don't want to leave them as raw integers that could cause the model to uncorrectly interpret a higher number as a greater magnitude rather than a seperate category).

I chose to use one-hot encoding here because none of the categories meant more than another for each variable, but instead each value represented a different discrete category with no natural ordering or ranking between them. One-hot encoding is able to create separate binary columns for each of the category values and treat everything independently, just as needed in this situation.

Here I utilized the general formatting from the code we looked at in class to implement the encoding.

3 GBDT

I trained a XGBClassifier as shown in class. It was trained with a learning rate of 0.1, max depth of 6, and up to 500 estimators, with early stopping triggered after 50 rounds of no improvement on the validation set.

3.1 Training vs Validation Loss

In the Training and Validation Loss Curve (Figure 5), both the train and validation curves decreased through all 500 estimators, with the best iteration being at estimator 499. This indicates the model has not converged within the given estimator budget, which means the model was still improving at the point training stoppped and would likely benefit from increasing n_estimators.

Additionally, the gap between the train and validation curves growing over time indicates overfitting, where the model is fitting to training data and therefore validation improvement stagnates. This could likely be improved with stronger regularization or a more shallow max_depth.

Figure 5: XGB Training vs Validation Loss Curve

3.2 Feature Selection

Feature importance was computed using the built in plot_importance function in XGBoost. The resulting plot can be seen in Figure 6.

In the figure it can be seen that EXT_SOURCE_MEAN was the dominant feature, confirming that the engineered mean of external credit scores was very informative. CODE_GENDER and NAME_EDUCATION_TYPE were also ranked highly, which showcase that demographic factors carry meaningful weight. Finally, engineered features such as PAYMENT_RATE and AGE_GROUP appear in the top 20 features, revealing these engineered features captured real predictive signals beyond the raw features (thus validating the chosen engineered features).

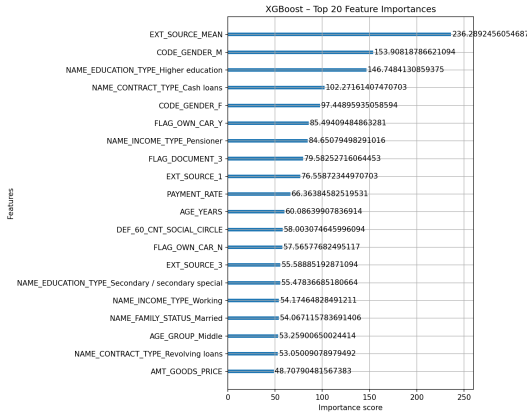

Figure 6: Feature Selection for XGB

3.3 Effect of Learning Rate

Three learning rates were compared: 0.01, 0.1, and 0.3 with all other parameters remaining fixed.

Through this experimentation, it was discovered that higher learning rates progressed quicker and none of the three rates led to a convergence, as shown in Figure 7.

Furthermore, learning rate 0.1 had the best validation AP of 0.249, followed by learning rate 0.01 with 0.244, and finally learning rate 0.3 being last with an AP of 0.215. This suggests that although learning rate 0.3 had the steepest initial descent, it moved too aggressively and overshot. Similarly, learning rate 0.01 can be seen to move too slowly and not maximize the estimator budget. Finally, learning rate 0.1 illustrated the best balance between speed and stability within the given constraints.

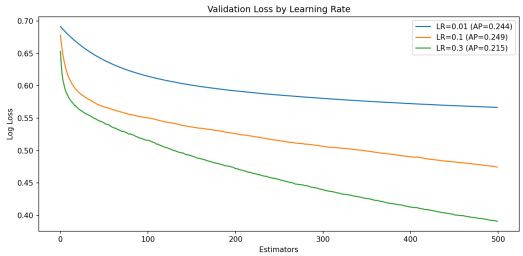

Figure 7: XGB Effect of Learning Rate

3.4 Other Parameters

Beyond learning rate, I also explored how other hyperparameters affected performance and overfitting.

For example, n_estimators was originally set to 500 with early stopping, but by estimator 499 it had not converged. Increasing this value would allow for further improvement. There is not enough space in this report to showcase the results, but here are the general takeaways compared to the current baseline results.

max_depth was originally set to 6, allowing for decently complex trees. Deeper trees reduce bias but increase variance. Based on the train-validation gap with this value, reducing it would likely reduce overfitting.

subsample was set to 0.9, randomly sampling 90% of training rows per tree. This decorrelates individual trees and acts as regularization. Lowering it further would increase this effect and could help close the train-validation gap.

reg_alpha and reg_lambda control the L1 and L2 regularization of leaf weights. reg_lambda=1.0 penalizes large leaf weights to discourage fitting noise. reg_alpha was left at 0, increasing it would introduce sparsity, which could be beneficial given the high dimensionality after one-hot encoding.

4 MLP

I trained an MLPClassifier using scikit-learn.

Before training, all features in the training set were standardized using StandardScaler fit, and then applied to the validation and test set. This standardization is required for MLP and not XGB because neural networks optimize weights using gradient descent, which is sensitive to the scale of input features while XGBoost makes decisions based on feature thresholds.

4.1 Training Loss Curve

The baseline MLPClassifier was trained with a (128, 64) architecture, ReLU activation, a learning rate of 0.001, and early stopping with a 10% internal validation fraction.

As shown in Figure 8, the loss decreases steadily and consistently throughout, dropping from 0.263 to 0.205, indicating stable learning without oscillation. However, the training stopped after only 12 iterations due to early stopping. This very short training window may indicate that the model stopped too early before fully exploring the loss landscape.

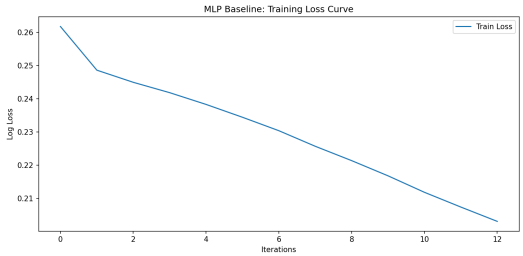

Figure 8: MLP Training Loss Curve

4.2 Effect of Hidden Layer Sizes

I compared three architectures: Shallow (64,), Medium (128, 64), and Deep (256, 128, 64). All other parameters were fixed.

As seen in Figure 9, all three models stopped within 13 iterations because of early stopping.

The Shallow network had the slowest initial descent, stagnates almost immediately, and has highest final loss. The AP for this model was 0.218, which suggests a single layer of 64 neurons did not have the capacity to capture the complexity of this dataset.

On the other hand, the Medium and Deep architectures were able to converge better. The Medium architecture achieved the best AP of 0.228 while the Deep architecture achieved the lowest final training loss. This showcases that the medium architecture is able to generalize better and the Deep network is able to fit the training data better. This also reveals that the Deep architecture likely had some overfitting.

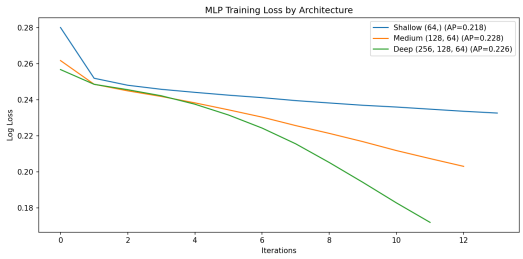

Figure 9: MLP Effect of Hidden Layer Sizes

4.3 Effect of Learning Rate

The learning rates of 0.001, 0.01, and 0.1 were compared with baseline configurations of (128, 64) architecture. The results can be seen in Figure 10.

Learning rate of 0.1 resulted in a generally flat, noisy loss curve that never had a steep decrease and a very poor AP of 0.084. This conveys that the learning rate here is too large, where the gradient updates were overshot at every step resulting in the model not being able to learn.

On the other hand, learning rate 0.001 had a much better loss curve. The curve had a steady, consistent decrease across all 12 iteration. It also had the best AP of 0.228, which indicates that a smaller learning rate allowed the model to navigate the loss language more precisely, ultimately resulting in the best overall predictive performance.

Lastly, the learning rate 0.01 was in the middle of the other two. It had a good initial steep descent in the loss curve, but converged and plateaued very quickly. This resulted in a higher final loss and lower AP value of 0.220. However, this showcases that this rate might be preferable in environments where training time is constrained since it was able to converge much quicker and only sacrificed a small amount of predictive performance.

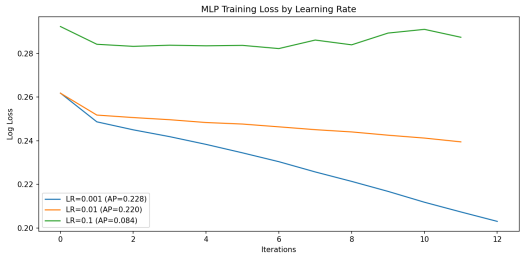

Figure 10: MLP Effect of Learning Rate

4.4 Effect of Activation

Two activation functions, ReLU and tanh, were compared with baseline configurations of (128,64) architecture and learning rate of 0.001.

ReLU achieved a validation AP of 0.228 while tanh achieved 0.213, making ReLU the better performing activation for this dataset.

This makes sense because ReLU avoids the vanishing gradient problem that tanh is susceptible to due to normalizing inputs into the range (-1,1). ReLU allows for the gradient values to be preserved, resulting in better weight updates. Since the MLP trains for very few iterations due to early stopping, the more efficient gradient flow of ReLU gives it a meaningful advantage in this setting.

4.5 Effect of max_iter

When tuning the parameters, for this model max_iter did not have much of an effect because early stopping would keep causing the training to halt when validation loss stopped improving. When I varied the max_iter from 20, 50, 100, and 200 there was no change because it was not a constraint. If the max_iter was chosen to be lower though, it would have been a risk for underfitting.

5 Comparison

XGBoost vs MLP — Test Set Metrics

Metric	XGBoost	MLP
Accuracy	0.773	0.9191
Precision	0.195	0.4259
Recall	0.5792	0.0062
F1-Score	0.2918	0.0122
AUC-PR	0.2414	0.223
Training Time	109.3405	53.0789

Figure 11: Summary Comparison Table

The two models, XGB and MLP, were evaluated side-by-side on the same test set across five metrics: Accuracy, Precision, Recall, F1-Score, and AUC-PR, along with training time. The full results are summarized in Figure 11.

At first glance, the MLP appears to be the stronger model. It achieves higher accuracy and higher precision. However, these numbers are misleading in the context of an imbalanced dataset. The most alarming is MLP's near-zero recall of 0.006, which illustrates that it correctly identified only 23 out of

3,724 true defaults in the test set, as confirmed by the confusion matrices in Figure 13. This reveals, the MLP learned to predict almost every applicant as non-default, exploiting the 91.9% majority class to achieve high accuracy without learning any meaningful signal about defaults.

On the other hand, XGBoost is more promising. While it had a lower accuracy, it was able to correctly identify 2,157 true defaults. A recall of 0.5792 demonstrates that it learned how to distinguish between the two classes. Additionally, its F1-Score of 0.292 is more than twenty times higher than the MLP's 0.012, and its AUC-PR of 0.241 exceeds the MLP's 0.223 (Figure 12). Therefore, in terms of this assignment, XGBoost is the better model.

Furthermore, XGBoost took approximately twice as long to train (109s vs 53s), which is expected given that boosting sequentially builds hundreds of trees. For a dataset of this size, both training times are reasonable, but the gap would widen significantly at larger scale.

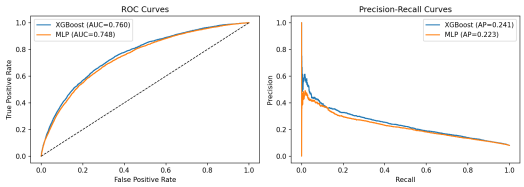

Figure 12: Comparison of ROC and PR Curves

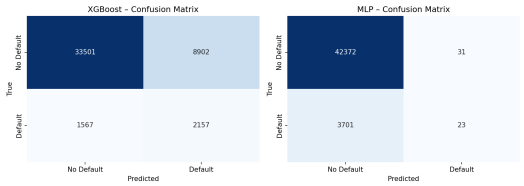

Figure 13: Comparison of Confusion Matrices

6 Discussion

6.1 When would you prefer GBDT over MLP, and vice versa?

GBDT is usually preferred for models that are using structured, tabular data with mixed features, imbalances, and non-linear relationships like this assignment. XGBoost's class weighting allowed for a better handling of the imbalances, especially compared to MLP.

MLP on the other hand is preferred when there is abundant data and features are dense and continuous, like images, text, or audio data. With data like that MLP is able to learn non-linear relationships over multiple layers which gives it an advantage over tree-based methods. Unfortunately, for this assignment which had tabular data, it is harder since it requires more careful tuning and is more sensitive to data processing decisions.

6.2 How does interpretability differ?

In terms of interpretability, the two models widely differ. XGBoost uses feature importance, while MLP is a black box.

Feature importance means that the XGBoost model quantifies each feature's contribution to a model's predictions. This provides key information for which variables contribute predictions, for example in this case which variables drive loan default risk. This interpretability is hence very open and helpful.

Black box, like what MLP uses, is conducted through a series of matrix multiplications and non-linear activations in hidden layers. Like the name suggests, it makes it really hard to understand why a particular prediction was made. However, it can lead to much more complex relationships and predictions.

6.3 How does each model handle categorical features and missing values?

XGBoost handles missing values natively by learning the optimal direction to send missing values at each split during training, meaning imputation is not necessary. In this assignment, imputation was still applied for consistency, but it was not required for XGBoost. For categorical features, XGBoost requires numeric input, so encoding was necessary, but it is relatively insensitive to the choice of encoding scheme since splits are threshold-based rather than distance-based.

The MLP requires significantly more preprocessing on both fronts. Missing values must be imputed before training since a neural network cannot process NaN values at all. Categorical features must also be encoded, and the choice of encoding can meaningfully affect performance. Most critically, the MLP requires feature scaling, which XGBoost does not. If any of these preprocessing steps are applied incorrectly, for instance, fitting the scaler on the full dataset rather than only the training set, data leakage can silently inflate performance estimates.

6.4 Which model is more sensitive to hyperparameter choices?

Both models are sensitive to hyperparameter tuning. However in this assignment, MLP was much more sensitive as seen by the differences due to learning rate or how training stopped early after 12 iterations.

6.5 Final Comments

Limitations: One of the key limitations for the results I presented was how the XGBoost model had no converged yet at 500 estimators, meaning that the output likely underestimate its true performance. If I had some more time I would run this again with a higher n_estimators.

Bias-Variance: XGBoost showcased high variance as seen by the large gap between training AUC and validation AUC. This is common with deep trees. On the other hand, MLP illustrated high bias with it's predictions, resulting in it predicting the majority class almost exclusively.

7 AI Tool Disclosure

I used Claude Code to help me in this assignment.

It's main contribution was in helping me generate detailed, informative graphs such as Figures 3-13. Claude was able to really speed up the process in formatting and structuring the visualizations I wanted, allowing me to focus spending time on the analytical and interpretive parts, rather than spending time on the exact formatting needed for matplotlib.

I also based some of my code from examples we saw in class in the provided Colab Notebooks. This includes the general structure of the one-hot encoding implementation using OneHotEncoder (from AML_L2_Colab_Demos.ipynb) and the GBDT structure (from AML_L5_Colab_Demos.ipynb).

Other than that, all the background information acquired, data cleaning justifications, feature engineering choices, analytical decisions, and written analysis in this assignment were contributed by me.